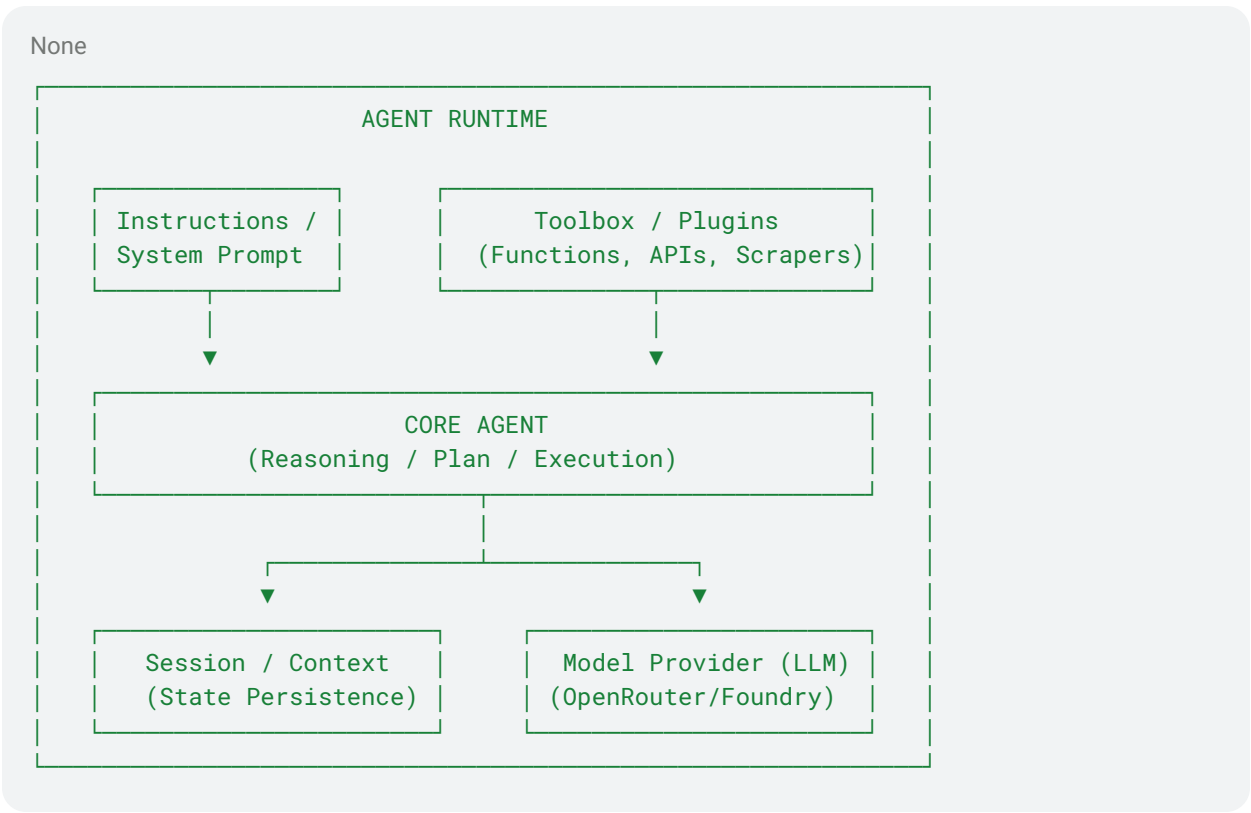


Guida Completa a Microsoft Agent Framework: Teoria, Architettura e Implementazione

Questa guida esplora il **Microsoft Agent Framework** (`Microsoft.Agents.AI` in .NET e `agent-framework` in Python), l'infrastruttura di nuova generazione di Microsoft progettata per unificare lo sviluppo di agenti autonomi, workflow multi-step e integrazioni con modelli di intelligenza artificiale.

1. Fondamenti Teorici e Concetti Chiave

Microsoft Agent Framework astrae la complessità dell'interazione con i Large Language Model (LLM) fornendo primitive modulari e componibili:



Le 4 Primitive Fondamentali

1. Agent (L'Entità Esecutrice):

- Un'unità autonoma che incapsula istruzioni di sistema (*metaprompt*), uno o più modelli LLM e una raccolta di strumenti (*Tools*).
- Riceve input dall'utente o da un altro agente, pianifica le azioni, invoca strumenti secondo necessità e sintetizza la risposta.

2. Tools & Function Calling:

- Metodi nativi del linguaggio (funzioni C# con attributi `[Description]` o funzioni Python tipizzate) esposti all'agente sotto forma di schemi JSON-Schema.
- L'agente non esegue direttamente codice arbitrario: il modello segnala la volontà di invocare un tool (`tool_call`), il framework esegue il metodo in locale in modo controllato e reinvia il risultato del tool al modello.

3. Sessions & State Management (Conversazioni Multi-Turn):

- La persistenza della memoria di conversazione. Il framework gestisce l'ordinamento dei messaggi (*User*, *Assistant*, *Tool*), la cronologia e la serializzazione dello stato di contesto.

4. Workflows & Multi-Agent Collaboration:

- Il framework supporta pipeline sequenziali, ad albero o guidate da messaggi dove l'output di un agente (es. *Discovery Scraper*) diventa l'input strutturato dell'agente successivo (es. *Copywriter*).

2. Integrazione con OpenRouter (Free Tier)

OpenRouter espone un'interfaccia standard **OpenAI-compatible**. Per connettere Microsoft Agent Framework a OpenRouter a costo zero, è sufficiente:

- Impostare l'endpoint personalizzato: `https://openrouter.ai/api/v1`
- Inserire l'API Key di OpenRouter (`sk-or-v1-...`)
- Selezionare un modello con suffisso `:free` (es. `meta-llama/llama-3.3-70b-instruct:free` o `qwen/qwen-2.5-72b-instruct:free`).

3. Implementazione in C# (.NET 8+)

3.1. Dipendenze NuGet

Nel file `.csproj`:

XML

```
<ItemGroup>
  <!-- Microsoft Agent Framework Core -->
  <PackageReference Include="Microsoft.Agents.AI" Version="*-*" />
  <!-- OpenAI Client standard per reindirizzare su OpenRouter -->
  <PackageReference Include="OpenAI" Version="2.1.0" />
</ItemGroup>
```

3.2. Esempio Completo: Lead Audit Agent con Tool Calling

C#

```
using System;
using System.ClientModel;
using System.ComponentModel;
using System.Threading.Tasks;
using OpenAI;
using Microsoft.Agents.AI;

namespace LocalLeadAgent
{
    // 1. Definizione dei Tools a disposizione dell'agente
    public class AuditTools
    {
        [Description("Analizza la presenza e lo stato tecnico del sito web di una specifica attività commerciale.")]
        public string AnalyzeWebsite(
            [Description("Il nome dell'attività da verificare")] string
            businessName,
            [Description("URL del sito web se presente, altrimenti stringa vuota")] string websiteUrl)
        {
            if (string.IsNullOrEmpty(websiteUrl))
            {
                return $"{businessName}: NESSUN sito web trovato sulla scheda Google Maps. Presenza solo telefonica.";
            }

            // Simulazione dei controlli eseguiti da Playwright
            bool isHttps = websiteUrl.StartsWith("https://",
            StringComparison.OrdinalIgnoreCase);
            bool isMobileFriendly = false; // Rilevato layout obsoleto non-responsive
            int copyrightYear = 2016;
```

```

        return $"{{businessName}} ({{websiteUrl}}): HTTPS={{isHttps}},
MobileFriendly={{isMobileFriendly}}, UltimoCopyright={{copyrightYear}}. Problema
rilevato: il sito non è navigabile da smartphone e non ha certificato SSL.";
    }
}

internal class Program
{
    static async Task Main(string[] args)
    {
        var openRouterKey =
Environment.GetEnvironmentVariable("OPENROUTER_API_KEY")
        ?? "sk-or-v1-tua-chiave-qui";

        // 2. Configurazione del client compatibile puntando a OpenRouter
        var openAiClient = new OpenAIClient(
            new ApiKeyCredential(openRouterKey),
            new OpenAIClientOptions
            {
                Endpoint = new Uri("https://openrouter.ai/api/v1")
            }
        );

        // 3. Creazione del Chat Client verso modello gratuito
        var chatClient =
openAiClient.GetChatClient("meta-llama/llama-3.3-70b-instruct:free");

        // 4. Inizializzazione dell'agente Microsoft Agent Framework
        var agent = new ChatClientAgent(chatClient)
        {
            Name = "LocalLeadAuditor",
            Instructions = """
Sei un consulente tecnico per attività locali dell'area
Jonico-Etna (Giarre, Acireale).
Il tuo compito è:
1. Usare il tool 'AnalyzeWebsite' per comprendere la situazione
web dell'attività target.
2. Redigere una bozza di email (Draft) professionale, breve e
priva di toni aggressivi.
3. Proporre una consulenza o un prototipo dimostrativo gratuito,
senza menzionare prezzi fissi.
            """
        };

        // Registrazione del Tool
        agent.Tools.Add(new AuditTools());

        // 5. Invocazione dell'agente (Multi-turn / Tool Calling automatico)

```

```

        var inputPrompt = "Prepara la bozza per 'Pasticceria Del Corso' ad
        Acireale. URL fornito: 'http://pasticceriadelcorso-vecchia.it'.";

        Console.WriteLine($"[Richiesta]: {inputPrompt}\n");
        Console.WriteLine("[Agente in elaborazione...]");

        var response = await agent.RunAsync(inputPrompt);

        Console.WriteLine("\n[Risposta Finale Generata dall'Agente]:");
        Console.WriteLine(response.Text);
    }
}

```

4. Implementazione in Python

4.1. Installazione

Shell

```
pip install agent-framework openai
```

4.2. Esempio Completo in Python

Python

```

import os
import asyncio
from openai import AsyncOpenAI
from agent_framework import Agent, tool

# 1. Definizione del Tool con type hints e docstring chiara
@tool
def analyze_lead_site(business_name: str, site_url: str = "") -> str:
    """Verifica lo stato online di un'attività locale e rileva eventuali problemi
    tecnici."""
    if not site_url:
        return f"{business_name}: Nessun sito esistente. L'attività perde clienti
        che cercano orari o servizi su mobile."

    # Dati simulati estratti da Playwright

```

```

    return (
        f"{business_name} - URL: {site_url}. "
        "Audit: Certificato SSL assente (HTTP semplice), "
        "layout privo di viewport mobile (zoom forzato da smartphone), "
        "copyright fermo al 2017."
    )

async def main():
    openrouter_key = os.getenv("OPENROUTER_API_KEY", "sk-or-v1-tua-chiave-qui")

    # 2. Configurazione client OpenAI-compatible puntato a OpenRouter
    openai_client = AsyncOpenAI(
        api_key=openrouter_key,
        base_url="https://openrouter.ai/api/v1"
    )

    # 3. Istanza dell'Agente con modello gratuito OpenRouter
    agent = Agent(
        client=openai_client,
        model="meta-llama/llama-3.3-70b-instruct:free",
        instructions=(
            "Sei un assistente specializzato in analisi web per commercianti della  

            provincia di Catania. "
            "Usa il tool a disposizione per verificare la presenza web. Poi genera  

            una bozza email di 4-5 righe: "
            "umile, concreta, territoriale e focalizzata su una soluzione pratica  

            per il cliente."
        ),
        tools=[analyze_lead_site]
    )

    prompt = (
        "Analizza l'attività 'Autofficina Jonica' a Giarre. "
        "Non hanno un sito registrato, hanno solo la scheda su Google Maps."
    )

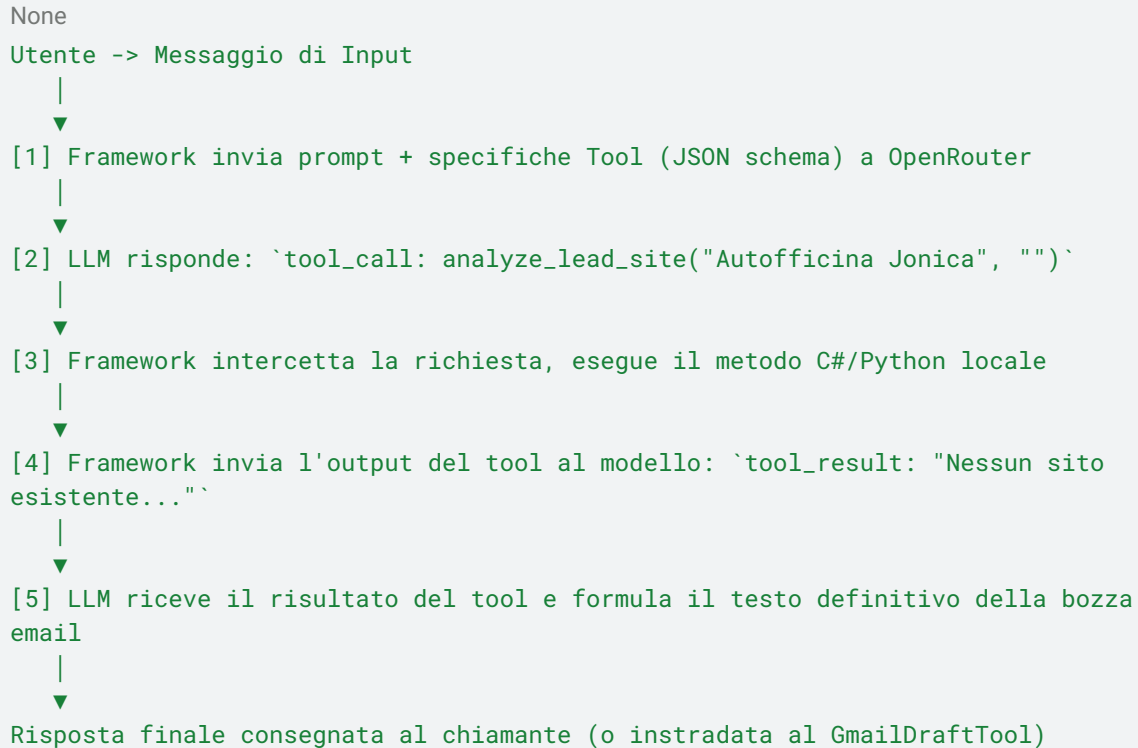
    print(f"Richiesta: {prompt}\n")
    response = await agent.run(prompt)
    print("Risposta Agente:\n")
    print(response.content)

if __name__ == "__main__":
    asyncio.run(main())

```

5. Meccanismo Interno: Ciclo ReAct (Reason + Act)

Dietro le quinte, il Microsoft Agent Framework gestisce il loop iterativo di reasoning e action:



6. Best Practice di Sviluppo

1. **Descrizioni dettagliate per i Tool:** I modelli gratuiti (come Llama 3.3 70B o Qwen 2.5 72B) dipendono fortemente dalla chiarezza della descrizione (`[Description]` o docstring) per capire quando e come invocare la funzione.
2. **Controllo degli Errori sui Tool:** Qualsiasi eccezione sollevata all'interno di un tool deve essere intercettata e restituita come messaggio di errore comprensibile al modello (es. *"Errore: Timeout durante la connessione al sito web"*), per permettere all'agente di adattare la risposta senza crashare.
3. **Controllo dello Stato (State Persistence):** Per flussi multi-step prolungati, memorizza gli ID dei lead già processati nel database SQLite locale prima di invocare l'agente, così da evitare chiamate ridondanti in caso di riavvio del processo.